Name: UID#: DirectoryID:

CMSC216: Practice Final Exam A

Spring 2026
University of Maryland

Exam period: 20 minutes *Points available:* 40

Problem 1 (10 pts): Pagebo Undary recently wrote a C program that is shown nearby and is startled to find that, despite his code clearly accessing out-of-bounds array indices, a Segmentation Fault does not occur unless the access is "way" out of bounds. Pagebo is confused by this apparent inconsistency but concludes that, so long as his code is only a "little" out of bounds, apparently nothing bad will happen.

```c
#include <stdio.h>
int main(){
  int arr[5]={10,20,30,40,50};
  printf("arr[   10]: %d\n",arr[   10]);
  printf("arr[  100]: %d\n",arr[  100]);
  printf("arr[10000]: %d\n",arr[10000]);
  return 0;
}
// >> gcc array_bounds.c
// >> a.out
// arr[   10]: 378735946
// arr[  100]: 1892438979
// Segmentation fault (core dumped)
```

Use your knowledge of the **Virtual Memory System** to educate Pagebo on why some out-of-bounds accesses generate Segmentation faults while others do not. Indicate whether you agree with Pagebo's conclusion (going a little out of bounds is okay) or if there is more to it than this.

Problem 2 (10 pts): New programmers are often surprised to learn that once an array is allocated, its size cannot be extended. In C code, this is easily observable as calling `malloc(16)` will yield a block of 16 bytes but there are no simple calls to expand this block of memory and calls like `realloc()` indicate they may move data to another location to find enough space.
Consider a proposed function for EL Malloc called `int el_expand_block(el_blockhead_t *bock)` which would expand a given block in place.

(A) What conditions need to occur for the function to succeed?
(B) Why is it impossible to expand a block in some cases?

Problem 3 (10 pts): Nearby is the output of `pmap` showing page table virtual memory mapping information for a running program called `memory_parts`. Answer the following questions about this output.

(A) Certain addresses of memory are marked with the annotation `r-x`. Explain what this means and what kind of information you would expect to find in those addresses.

(B) Why does `pmap` only show a limited number of virtual addresses? What would happen if the program attempted to access an address not listed in the output? Example: address `0x00` is not in the listing.

```
>> pmap 7986
7986:   ./memory_parts
00005579a4abd000      4K r-x-- memory_parts
00005579a4cbd000      4K r---- memory_parts
00005579a4cbe000      4K rw--- memory_parts
00005579a4cbf000      4K rw---   [ anon ]
00005579a53aa000    132K rw---   [ heap ]
00007f441f2e1000   1720K r-x-- libc-2.26.so
00007f441f48f000   2044K ----- libc-2.26.so
00007f441f68e000     16K r---- libc-2.26.so
00007f441f692000      8K rw--- libc-2.26.so
00007f441f694000     16K rw---   [ anon ]
00007f441f698000    148K r-x-- ld-2.26.so
00007f441f88f000      8K rw---   [ anon ]
00007f441f8bb000      4K r---- gettysburg.txt
00007f441f8bc000      4K r---- ld-2.26.so
00007f441f8bd000      4K rw--- ld-2.26.so
00007f441f8be000      4K rw---   [ anon ]
00007fff96ae1000    132K rw---   [ stack ]
00007fff96b48000     12K r----   [ anon ]
00007fff96b4b000      8K r-x--   [ anon ]
 total             4276K
```

Problem 4 (5 pts): Pyra Midmem read in a free online blog post "Memory for Morons" that there is no need to invest much money in buying DRAM. Instead, one can configure the operating system's Virtual Memory system to use Permanent Storage disk drives as Main Memory leading to a much less expensive computer with a seemingly large memory. Pyra is quite excited about this as some programs she wants to run need a lot of main memory to run fast and these days DRAM ain't cheap. Advise her on any risks or performance drawbacks she may encounter using such "low-RAM" approach.

Problem 5 (5 pts): Ripsep Arate is trying to understand how each thread executes potentially different instructions. Processes make sense to him: they have their own Text region with different code. But Threads all share the same Text region so how can they behave differently? Explain to Ripsep how this is possible. Reference relevant hardware that controls the code a Thread will execute.